

Grocery Compare

Sprint plan and documentation checklist

Compares Checkers, Pick n Pay, Spar and Woolworths so people find the cheapest store, or mix of stores, for their whole list. Front end is three files: index.html, style.css, script.js.

Decisions locked in

Stores and categories	Food items plus household and convenience items (cleaning and everything a household spends on monthly)
Prices and location	Depend on the branch closest to the user or the branch they select
Accounts	None. The saved list lives in the browser (localStorage)
Price refresh	Every 2 weeks, with the last-updated date shown to the user
Specials	Loyalty card specials shown next to the original price in a different color, and clearly labeled

Risks to settle early

- **Price source.** Real shelf prices for four retailers by branch need a legitimate source (feeds, licensed data, partnerships, or manual capture). Check each retailer's terms before collecting anything automatically. Until then the app runs on sample data.
- **Product matching.** Matching the same product, brand and size across stores matters more than catalogue size.
- **Branch-level data.** Branch prices multiply the data volume, so Sprint 5 may start with price zones if per-branch data is not available.

Overview

Sprint	Focus	Area
0	Repo and project setup	SETUP
1	Front-end foundation	UI
2	Product grid and controls	CORE
3	Shopping list and totals	CORE
4	Catalogue and data model	DATA
5	Branch and location	DATA
6	Loyalty card specials	UX
7	Price updates and freshness	DATA
8	Quality, accessibility and performance	QA
9	Launch and handover	LAUNCH

SETUP

Sprint 0: Repo and project setup

Get the collaborators working from one clean, shared starting point.

- ☐ Create the git repository and push an initial commit
- ☐ Add README.md (what the app is, how to run it, folder guide)
- ☐ Add .gitignore and a LICENSE decision
- ☐ Agree on branch strategy (main protected, feature branches, pull requests)
- ☐ Agree on commit message style and code review rules
- ☐ Create the folder structure: index.html, style.css, script.js, /data, /docs
- ☐ Add the UX/UI guide to /docs and this sprint plan to /docs
- ☐ Create the issue board with one issue per checklist item

Sprint review notes: _____

UI

Sprint 1: Front-end foundation

Page structure and the design system, with no behavior yet.

- ☐ index.html structure: top bar, hero, controls, product grid, list panel, footer
- ☐ CSS variables for all tokens (Ember Orange, Verdant Green, Graphite, Fog, Paper White, store dots)
- ☐ Load DM Sans (500, 600, 700); type scale 60 / 22 / 17 / 12 to 14 px
- ☐ 26px radius on every card, button, chip and input; 8px spacing steps; 1200px max width
- ☐ No shadows and no gradients; no colored text on colored backgrounds
- ☐ Sticky top bar with logo and My list button with count badge
- ☐ Hero with two-tone headline (green "Compare") and the main search box
- ☐ Mobile-first layout that expands to a two-column desktop layout (sticky list panel)
- ☐ Dark mode through CSS variables following the device setting

Sprint review notes: _____

CORE

Sprint 2: Product grid and controls

Users can find products and see prices side by side.

- ☐ Product data array loaded at the top of script.js with sample data in the agreed shape
- ☐ Product card: name, size, category, plus button, price row per selected store
- ☐ Unit price in small text (price divided by d, with u label)
- ☐ Lowest price highlighted in green; "Save up to R..." line
- ☐ Live search filtering as the user types
- ☐ Category chips: All plus every launch category
- ☐ Sort dropdown: Name A to Z, Biggest saving, Lowest price
- ☐ Store chips: choose stores to compare; at least one always stays selected
- ☐ Empty state when no products match

Sprint review notes: _____

CORE

Sprint 3: Shopping list and totals

The core question answered: where is my whole list cheapest?

- ☐ Add, remove and change quantity for list items
- ☐ Instant total per store with the cheapest store marked
- ☐ Extra cost versus the cheapest shown for every other store
- ☐ "Split your shop": total if each item is bought at its cheapest store, with what to buy where
- ☐ Saved list in the browser with localStorage (survives refresh)
- ☐ Copy list button: list plus store totals as plain text
- ☐ My list button scrolls to the list panel; badge count stays in sync
- ☐ List panel moves below the products on phones

Sprint review notes: _____

Sprint 4: Catalogue and data model

Cover food and monthly household spend, with correct product matching.

- ☐ Define launch categories: food (Dairy, Bakery, Pantry, Meat, Fruit and Veg, Frozen, Drinks) and household (Cleaning, Laundry, Toiletries, Paper goods, Baby, Pet)
- ☐ Extend the product schema: barcode, brand, and per-store optional prices
- ☐ Product matching rule: same barcode, or same brand, type and size
- ☐ Handle missing prices: skip the store for that item and show "Not available" in card() and the totals code
- ☐ Totals and split logic only count stores that stock every list item (or flag the gaps clearly)
- ☐ Seed the catalogue with realistic sample products for every category
- ☐ Validation script that checks the data file (unique ids, valid categories, no bad prices)
- ☐ Document the data format in /docs for whoever supplies prices

Sprint review notes: _____

Sprint 5: Branch and location

Prices depend on the branch closest to the user or the one they pick.

- ☐ Branch list per store: name, area, coordinates
- ☐ Price data keyed by branch (or by a region/price-zone if branches share pricing)
- ☐ Branch picker per store, with search by suburb or town
- ☐ "Use my location" option to find the nearest branch of each store (browser geolocation, permission asked politely)
- ☐ Fallback when location is denied: manual picker and a sensible default
- ☐ Save the chosen branches in localStorage
- ☐ Show the selected branch name under each store in the price rows and totals
- ☐ Handle a store with no branch nearby

Sprint review notes: _____

UX

Sprint 6: Loyalty card specials

Show special pricing clearly next to the normal price.

- ☐ Add optional special fields per price: special amount, loyalty program name, valid until
- ☐ Show the special price next to the original price in a distinct color
- ☐ Choose the special color so it does not clash with green (cheapest) or orange (actions)
- ☐ Label the card type (for example Smart Shopper, Xtra Savings, Rewards, WRewards) so people know what they need
- ☐ Toggle: "I have this store's loyalty card" per store, to include or exclude specials in totals
- ☐ Totals, cheapest store and split shop respect the loyalty toggle
- ☐ Explain loyalty pricing in a short note on the page
- ☐ Hide expired specials automatically

Sprint review notes: _____

DATA

Sprint 7: Price updates and freshness

Prices refresh every 2 weeks and the user can see how fresh they are.

- ☐ Decide and document the price source (partner feed, licensed data, or manual upload) and check the terms of use
- ☐ Write an import script that turns the source into the data file
- ☐ Schedule the refresh every 2 weeks (cron or CI scheduled job)
- ☐ Store a lastUpdated date with the data
- ☐ Show "Prices last updated: " under the controls
- ☐ Warn visually if the data is older than 14 days
- ☐ Keep a changelog of each refresh so a bad import can be rolled back
- ☐ Notice that specials may change between refreshes

Sprint review notes: _____

QA

Sprint 8: Quality, accessibility and performance

Make it solid before anyone else uses it.

- ☐ Test on common phone sizes and desktop widths (Chrome device toolbar plus real phones)
- ☐ Test on Chrome, Safari, Firefox and Edge
- ☐ Keyboard navigation and visible focus states
- ☐ Screen reader labels for the plus, quantity and chip buttons
- ☐ Color contrast check for both light and dark mode
- ☐ Performance check with the full catalogue (fast search and sort)
- ☐ Edge cases: empty list, duplicate adds, huge quantities, cleared localStorage, offline
- ☐ Money formatting and rounding checks (pounds and cents)
- ☐ Bug triage and fixes

Sprint review notes: _____

LAUNCH

Sprint 9: Launch and handover

Put it online and make it easy to maintain.

- ☐ Choose hosting and deploy (a static host is enough)
- ☐ Custom domain and HTTPS
- ☐ Simple privacy note (list and branch kept on the user's device only)
- ☐ Basic analytics decision (yes/no, privacy friendly)
- ☐ Maintenance guide: how to refresh prices, add a store, add a category
- ☐ Tag release v1.0.0 in git
- ☐ Retrospective and backlog for later: price history, price drop alerts, accounts if ever needed

Sprint review notes: _____